

ESP-IDF Development Setup Guide

Installing and configuring ESP-IDF with Visual Studio Code on Windows, and building your first blink project on an ESP32-C6.

1. Prerequisites

- **Git** — needed to clone and manage the ESP-IDF repository
- **Visual Studio Code** — the IDE used throughout this guide

2. Install the ESP-IDF Extension

- Open VS Code and go to **Extensions** (Ctrl+Shift+X)
- Search for "**ESP-IDF**"
- Install the official extension published by **Espressif Systems**

3. Install the ESP-IDF Framework and Toolchain

Using ESP-IDF's official PowerShell installer script (`install.ps1`), which downloads the toolchain, clones the framework, and builds the Python virtual environment.

3.1 — Set the tools path

Open PowerShell and set where toolchains and the Python environment will live:

```
$env:IDF_TOOLS_PATH = "C:\Espressif\tools"
```

3.2 — Run the installer

Navigate to your ESP-IDF clone and run the install script:

```
cd C:\esp\v6.0.2\esp-idf
.\install.ps1
```

3.3 — Verify the installation

```
.\export.ps1
idf.py --version
```

If this prints a version number, the ESP-IDF environment (including the Python virtual environment) was built successfully.

4. Point the VS Code Extension at Your Installation

Open your VS Code user settings file directly:

```
code $env:APPDATA\Code\User\settings.json
```

Add the following block inside the existing outer { } of the file:

```
"idf.customExtraVars": {  
  "IDF_PATH": "C:\\esp\\v6.0.2\\esp-idf",  
  "IDF_TOOLS_PATH": "C:\\Espressif\\tools",  
  "IDF_PYTHON_ENV_PATH": "C:\\Espressif\\tools\\python_env\\idf6.0_py3.10_env"  
}
```

The Python virtual environment path must point to the folder actually created by install.ps1 (found under tools\\python_env\\), not an EIM-style path.

Save the file, then reload the window: Command Palette (Ctrl+Shift+P) → **Developer: Reload Window**.

4.1 — Trust the workspace

The first time a folder is opened in VS Code, a banner may appear reading *"Restricted Mode"*. Click it and choose **"Trust the authors"** — required for the ESP-IDF extension's commands to activate.

4.2 — Verify the extension is working

Command Palette → **ESP-IDF: Doctor Command**. This runs a full diagnostic and confirms the extension can see your ESP-IDF path, tools, and Python environment correctly.

5. Create a Project (outside the framework folder)

Important: the ESP-IDF framework folder itself (e.g. C:\\esp\\v6.0.2\\esp-idf) is not a buildable project — it's the SDK your projects build against. Projects must live in a separate folder.

Copy an example project into its own folder:

```
Copy-Item -Path "C:\\esp\\v6.0.2\\esp-idf\\examples\\get-started\\hello_world" ` -Destination "C:\\esp\\projects\\hello_world" -Recurse
```

Then open that new folder as the active workspace: **File** → **Open Folder** → select C:\\esp\\projects\\hello_world, and let VS Code reload.

6. Configure the Project

- Command Palette → **ESP-IDF: Set Espressif Device Target** → select your chip (e.g. esp32c6)
- Plug in the board, then Command Palette → **ESP-IDF: Select Port to Use** → choose the detected COM port

7. Build, Flash, and Monitor

- **ESP-IDF: Build your Project** — compiles the firmware
- **ESP-IDF: Flash your Project** — uploads the compiled binary to the board
- **ESP-IDF: Monitor your Device** — opens the live serial log
- Or run all three in one step: **ESP-IDF: Build, Flash and Monitor**

On first successful run, the ESP32-C6 printed "Hello world!", chip information, and a countdown/restart loop confirming the environment was fully functional end-to-end.

8. Customizing the Blink Example for a Specific GPIO Pin

To drive a plain LED on a specific GPIO pin directly (bypassing the addressable LED-strip / Kconfig logic), two files were edited inside the project.

8.1 — main/hello_world_main.c

Includes required at the top of the file:

```
#include <stdint.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
```

Core logic driving the LED on a hardcoded GPIO pin:

```
#define BLINK_GPIO 1

static uint8_t s_led_state = 0;

static void configure_led(void)
{
    ESP_LOGI(TAG, "Example configured to blink GPIO LED!");
    gpio_reset_pin(BLINK_GPIO);
    gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);
}

static void blink_led(void)
{
    gpio_set_level(BLINK_GPIO, s_led_state);
}

void app_main(void)
{
    configure_led();

    while (1) {
        ESP_LOGI(TAG, "Turning the LED %s!", s_led_state == true ? "ON" : "OFF");
        blink_led();
        s_led_state = !s_led_state;
    }
}
```

```
        vTaskDelay(1000 / portTICK_PERIOD_MS);  
    }  
}
```

8.2 — main/CMakeLists.txt

The GPIO driver component must be declared as a dependency so its headers are made available to the compiler. In ESP-IDF v6.0, the legacy "driver" component was split apart, and GPIO now has its own component name: `esp_driver_gpio`.

```
idf_component_register(SRCS "hello_world_main.c"  
                      PRIV_REQUIRES spi_flash esp_driver_gpio  
                      INCLUDE_DIRS "")
```

The `#include` path in the C file still reads `"driver/gpio.h"` even though the component itself is named `esp_driver_gpio` — only the `CMakeLists.txt` dependency name changed.

8.3 — Rebuild and flash

Command Palette → **ESP-IDF: Build, Flash and Monitor**. The LED on the configured GPIO pin blinks on and off every second, confirmed by the alternating "Turning the LED ON!" / "Turning the LED OFF!" log messages.

End of guide — ESP32-C6 blink example running successfully via ESP-IDF v6.0.2 in Visual Studio Code.